



Ing. Fernando Negozi

fernando.negozi@eletica.it

www.eletica.it



history

serve per visualizzare lo storico dei comandi inseriti

Sintassi: `history [num]`

num: numero intero

AD ESEMPIO:

`history` - Stampa tutti i comandi inseriti in precedenza

`history 1` stampa l'ultimo comando che è stato inserito (cioè History nella forma: *N history 1*)

Per avere quello precedente ad history: history 2

more

serve per visualizzare il contenuto di un file o dello standard input una pagina alla volta

Sintassi: *more [option] [file]*

file: file di testo da visualizzare

Opzioni:

- n numero di righe da visualizzare in una pagina.
- " numero " numero riga di inizio della visualizzazione
- p visualizza il prompt "More? " al termine di ogni pagina in attesa di istruzioni
- s raggruppa le linee vuote

E' possibile scorrere il contenuto in avanti con la barra spaziatrice

Per uscire: q

ATTENZIONE: non permette di scorrere indietro

less

serve per visualizzare il contenuto di un file o l'output di un altro comando, una pagina alla volta

Sintassi: *less [option] [file]*

file: file di testo da visualizzare

Opzioni:

- n -N mostra il numero di riga in alto a sinistra di ogni pagina
- i -I ignora la differenza tra maiuscole e minuscole
- F esce automaticamente se la visualizzazione occupa meno di una pagina di testo
- S tronca le linee quando superano la larghezza della finestra
- e -E esce al termine del file

E' possibile scorrere il contenuto in avanti e indietro utilizzando le frecce direzionali

E' possibile cercare una stringa all'interno del file digitando **"/stringa da cercare"** o **"?stringa da cercare"**

Per uscire: q

Esempio - Ricerca di un pattern

- *Spostarsi nella directory /proc*
- *Aprire il file devices con less*
- *Cercare un pattern come ad esempio 'usb' utilizzando /o ?*

cat

Visualizza il contenuto di uno o più file di testo concatenati in uno stesso output sullo standard output.

Sintassi: *cat [OPTION]... [FILE]...*

[FILE]... uno o più filer da visualizzare e concatenare , se non presenti preleva da standard input

Opzioni:

- n per numerare le righe di output
- b numera solo le righe non vuote
- s raggruppa le righe vuote
- E mostra il carattere di fine riga
- T mostra i caratteri di tabulazione

`cat eserc1.txt eserc2.txt` #legge il contenuto dei file, li concatena e visualizza sullo standard output.

tr

Trasforma o elimina caratteri dallo standard input verso lo standard output

Sintassi: *tr [OPTION]... SET1 [SET2]*

SET1 : insieme di caratteri da trasformare

SET2: caratteri di destinazione

Opzioni:

- c complementa l'insieme di caratteri specificato
- d elimina tutti i caratteri presenti in set1

Standard POSIX (definisce le classi di caratteri - formato [: :])

[:alnum:] - Tutte le lettere e i numeri

[:alpha:] - Tutte le lettere

[:blank:] - Tutti gli spazi bianchi orizzontali

[:cntrl:] - Tutti i caratteri di controllo

[:digit:] - Tutti i numeri

[:graph:] - Tutti i caratteri stampabili, escluso lo spazio

[:lower:] - Tutte le lettere minuscole

[:print:] - Tutti i caratteri stampabili, incluso lo spazio

[:punct:] - Tutti i caratteri di punteggiatura

[:space:] - Tutti gli spazi bianchi orizzontali o verticali

[:upper:] - Tutte le lettere maiuscole

[:xdigit:] - Tutti i numeri esadecimali



esercizio

Scrivi uno script bash che legge una stringa e la converta tutta in maiuscolo

Utilizzare: tr



esercizio - tr

#!/bin/bash

read -p "Inserisci la stringa: " input

output=\$(echo "\$input" | tr '[:lower:]' '[:upper:]')

echo "La stringa convertita è: \$output"

#legge la stringa

#tr converte



Esercizio (tr_1.sh)

*Scrivi uno script bash che legge il nome di un file di testo e converta le vocali nel file in **

Utilizzare: tr



Esercizio (tr_1.sh)

```
#!/bin/bash
```

```
read -p "Inserisci il nome del file di testo: " filename
```

#Leggi il nome del file di testo

```
if [[ -f "$filename" && -r "$filename" ]];
```

#Controlla se il file esiste e se è leggibile

```
then
```

```
output=$(cat "$filename" | tr 'aeiouAEIOU' '*')
```

#sostituire tutte le vocali (maiuscole e minuscole) con *

```
echo "Il testo modificato è: "echo "$output"
```

#Stampa il testo

```
else
```

```
echo "Il file $filename non esiste o non è leggibile."
```

```
fi
```

grep (global regular expression print)

cerca una stringa o un'espressione regolare all'interno di uno o più file

Sintassi: *grep [option] pattern [file1 file2 ...]*

file1 file 2 .. : file in cui cercare

Pattern: espressione da cercare

Opzioni:

- i per effettuare una ricerca case-insensitive
- v ricerca inversa, tutte le linee che NON corrispondono al pattern
- r per effettuare una ricerca ricorsiva all'interno di una directory
- e ricerca per più pattern inseriti

es:

```
grep hello esercizio.sh
grep -i 'hello world' menu.h
grep [aeiou] esercizio.txt
```

cerca la stringa '-v' nel file.txt:

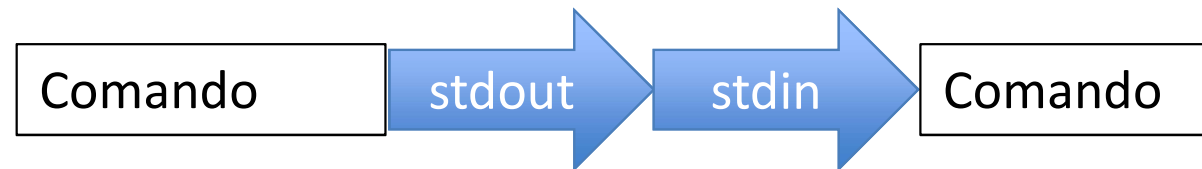
```
grep -e -v esercizio.txt
```

Cerca pat1 e pat2 in esercizio.txt:

```
grep -e pat1 -e pat2 esercizio.txt
```

PIPELINE

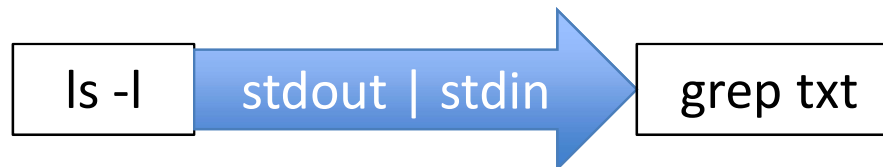
Simbolo: |



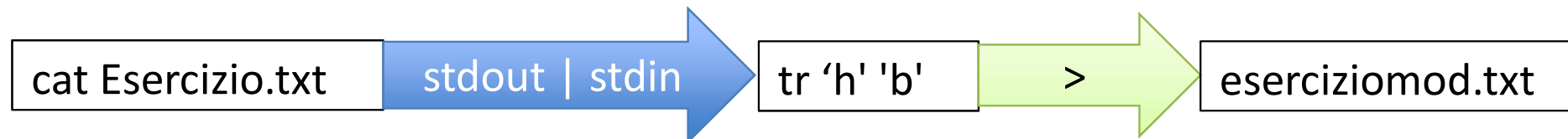
La *pipeline* ridirige lo *standard output* del comando a sinistra del simbolo nello *standard input* del comando di destra.

Esempi:

`ls -l | grep txt`



`cat Esercizio.txt | tr 'h' 'b' > serciziomod.txt`



Si possono utilizzare più pipeline in cascata:



cat Esercizio.txt | grep hello | grep world

Risultato diverso da: *cat Esercizio.txt | grep 'hello world'*

grep estrae dal file prima le linee che contengono *'hello'* e tra queste linee estrae quelle che contengono *'world'*

ESER 1

- Fare un elenco dei file e delle directory contenute in / in un file di nome: *contenuto_root*.
Non elencare le sottodirectory delle directory di /

ESER 2

- Stampare il manuale del comando *ls* in fondo al file *"contenuto_root"*
- Cercare all'interno del file la stringa "COPYRIGHT"

ESER 3

- Stampare il manuale di *ls* a video utilizzando una pipeline e cercare la stringa "COPYRIGHT"



ESER 1

- Fare un elenco dei file e delle directory contenute in / in un file di nome: *"contenuto_root"*.
Non elencare le sottodirectory delle directory di /

```
touch contenuto_root  
ls / > contenuto_root
```

ESER 2

- Stampare il manuale del comando *ls* in fondo al file *"contenuto_root"*
- Cercare all'interno del file la stringa "COPYRIGHT"

```
man ls >> contenuto_root  
less contenuto_root  
lanciare il comando /COPYRIGHT
```

ESER 3

- Stampare il manuale di *ls* a video utilizzando una pipeline e cercare la stringa "COPYRIGHT"

```
man ls | less  
Lanciare il comando /COPYRIGHT
```



ESER 4

- Analisi del comando

```
$ pstree > pstree1 && more pstree1
```



ESER 4

- Analisi del comando

```
$ pstree > pstree1 && more pstree1
```

Esegue il comando `pstree` ,

ridirige l'output nel file `pstree1` – ora presente nella directory

poiché il comando è stato eseguito con successo stampa a video il file `pstree1`

Se si realizza uno script che deve essere richiamato fornendogli degli argomenti (dei parametri) sotto forma di opzione, come si è abituati con i programmi di servizio comuni, può essere conveniente l'utilizzo di programmi o comandi appositi.

Tradizionalmente si fa riferimento a **getopt**
ha la sintassi:

getopt stringa_di_opzioni parametro...

La stringa di opzioni è un elenco di lettere che rappresentano le opzioni ammissibili; se ci sono opzioni che richiedono un argomento, le lettere corrispondenti di questa stringa devono essere seguite dal simbolo due punti (:). Gli argomenti successivi sono i valori dei parametri da analizzare.

Es: *getopt ab:c -a uno -b due -c tre quattro[Invio]*

comando `getopt` per analizzare le opzioni da riga di comando.

```
STRINGA_ARGOMENTI=$(getopt -o ab:c -- "$@")
```

Dove:

- o specifica le opzioni valide per lo script
- : si mettono dopo le opzioni che richiedono un argomento.
- indica la fine delle opzioni
- "\$@" rappresenta tutti gli argomenti passati allo script.

Il risultato viene salvato nella variabile `STRINGA_ARGOMENTI`



(getoption_1.sh)

```
#!/bin/bash
# scansione_1.sh
# Si raccoglie la stringa generata da getopt.
STRINGA_ARGOMENTI=$(getopt -o ab:c -- "$@")
```

```
# Si trasferisce nei parametri $1, $2,...
eval set -- "$STRINGA_ARGOMENTI"
while true; do
case "$1" in
-a) echo "Opzione a"
    shift
    ;;
-b) echo "Opzione b, argomento $2"
    shift 2
    ;;
-c) echo "Opzione c"
    shift
    ;;
--)
    shift
    break
    ;;
*) echo "Errore non previsto"
    exit 1
    ;;
esac
done
```

```
echo "Argomenti rimanenti:"
for argomento in "$@"
do
echo "$argomento"
done
```